

Institut Supérieur d'Informatique et de Mathématiques de Monastir

Cours :Architecture des Systèmes d'Entreprise (ESA)

Chapitre3: Styles & Modèles Architecturaux Usuels

Enseignante: Dr. Aljia BOUZIDI

aljia.bouzidi95@gmail.com

3ème Licence en Sciences d'Informatique

Année Universitaire :2025-2026

Objectifs

- Connaitre les modèles architecturaux utilisés dans la conception de la vue physique et la vue logique d'un système d'entreprise les plus connus
- Apprendre le fonctionnement des modèles architecturaux usuels
- Savoir faire la conception en se basant sur les patrons architecturaux

Contenu du chapitre 3

1. **Partie 1 :** Introduction
2. **Partie 2 :** Modèles Architecturaux de la vue physique
3. **Partie 3** : Modèles Architecturaux de la Vue Logique :

   La Famille MV*



Institut Supérieur d'Informatique et de Mathématiques de Monastir

Partie 1: Styles Architecturaux

Contenu de la Partie 1

1. Mise en contexte
2. Styles, patrons et idiomes
3. Qu'est ce un Style Architectural?
4. Architecture en Couches
5. Architecture orienté objet
6. Architecture orienté agents

Mise en contexte

- **L'architecture logicielle** peut se catégoriser en styles:
- En effet, malgré les millions de systèmes informatiques construits par le monde au cours des cinquante dernière années, tous se classent parmi un nombre extrêmement restreint de style architecturaux;
- De plus, un système informatique peut **utiliser plusieurs styles selon** le niveau ***de granularité*** ou **l'aspect du système** décrit.

Styles, patrons et idiomes (1/2)

- **Termes fréquemment utilisés pour décrire:**
  - Les structures utilisées pour la conception de logiciel
  - Des recettes éprouvées et reconnues des experts
  - Des exemples de bonnes pratiques (Par opposition aux mauvaises pratiques (anti-patterns))
  - Des trucs et astuces
  - Des spécificités utiles de certaines technologies

Styles, patrons et idiomes (2/2)

- **Styles (macro patterns)**
  - Structures de haut niveau pour décrire un logiciel d'un point de vue global
  - Ex. Modèle par couche (layered), MVC, …

- **Patrons (micro patterns)**
  - Solutions abstraites applicables à différents contextes mais qui offrent les mêmes avantages chaque fois
  - Correspond aux Design Patterns

- **Idiomes**
  - Décrit de bonnes pratiques liées
    - à un formalisme de programmation
    - à un langage en particulier

Qu'est ce un Style Architectural?

- Un **style architectural** est un ensemble de **principes** et de **lignes directrices** qui définissent la façon dont **les composants d'un système logiciel** doivent être organisés, interagir et communiquer entre eux.

- Il s'agit d'un **cadre général** qui permet de **créer une structure cohérente** pour le **système** en utilisant des schémas d'interaction et des conventions spécifiques.

- Les **styles architecturaux fournissent** des ***solutions abstraites*** à des problèmes communs de conception logicielle.

- Les **styles architecturaux sont des approches générales de conception logicielle** qui définissent des principes et des schémas d'interaction

- Exemple de style architecturaux usuels : Architecture en couche (loyered architecture), Architecture orienté objet, Architecture orienté service (SOA), Architecture orienté agents, Architecture orienté données, architecture orienté évènements, etc.

Architecture en Couches (1/2)

- **L'architecture en couches** se réfère généralement à la **manière dont un système logiciel** est structuré **en couches distinctes**, chacune ayant une responsabilité spécifique et interagissant avec les couches adjacentes de manière bien définie.

- L'idée principale derrière l'architecture en couches est **de diviser un système complexe en parties** plus gérables et modulaires.
- Chaque couche se concentre sur une tâche ou une fonction particulière,

- et les interactions entre les couches sont définies par des **interfaces claires** et **des protocoles de communication standardisés**.

Architecture en Couches (2/2)

- Cela rend le **développement, la maintenance et la mise à jour du système plus efficaces**, car chaque **couche** peut être **développée** et **testée indépendamment des autres**.
- Exemple d'architecture en couche:
  - Architecture des tiers (1_tier, 2_tiers, …)
  - La famille de MV_ (MVC, MVVM, MVP, MVT, etc…)

Architecture orienté objet (1/2)

- Les composants du système (**objets**) intègrent des **données** et des **opérations** de traitement de ces données.
- La **communication** et la coordination entre les objets sont réalisées par un mécanisme **de passage de messages**.
- **L'architecture orientée objets** est souvent décrite par les **trois piliers**:
  - **Encapsulation**
  - **Héritage**
  - **Polymorphisme**

Architecture orienté objet (2/2)

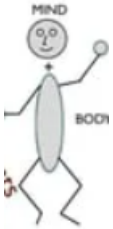
- **Encapsulation** concerne l'architecture détaillée de chaque objet, les données étant protégées d'accès direct par une couche d'interface. De plus, les sous-fonctions ,inutiles pour utiliser l'objet, sont masquées à l'utilisateur de l'objet

- **L'héritage** permet d'éviter la redondance de code et facilite l'extensibilité du logiciel, les fonctionnalités communes à plusieurs classes d'objets étant regroupées dans un ancêtre commun

- **Le polymorphisme** permet d'utiliser des objets différents (possédant des comportements distincts) de manière identique, cette possibilité est réalisée par la définition d'interfaces à implémenter (classes abstraites)

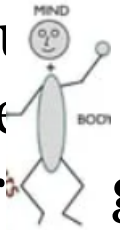
Architecture orienté agents (1/2)

- **L'architecture orientée agents** est un style d'architecture logicielle basée sur les agents ce qui correspond à un paradigme où l'objet, de composant passif, devient un composant projectif **:**
  - Dans la conception objet, **l'objet est essentiellement un composant passif,** offrant des services, et utilisant d'autres objets pour réaliser ses fonctionnalités



Architecture orienté agents (2/2)

- L'agent logiciel, contrairement à un objet, utilise de manière **relativement autonome**, avec une **capacité d'exécution propre**, **les autres agents pour réaliser ses objectifs** :
  - Il établit des dialogues avec les autres agents,
  - il négocie et échange de l'information,
  - Il décide à chaque instant avec qui agents communiquer en fonction de ses besoins immédiats et des disponibilités des autres agents.





Institut Supérieur d'Informatique et de Mathématiques de Monastir

Partie 2: Modèles Architecturaux de la vue physique

Contenu de la Partie 2

1. Vue Physique et Architecture en Couches
2. Architecture en Couches
3. Architecture 1-tier: Mainframe
4. Architecture 2-tiers: Client/Serveur
5. Architecture 3-tiers
6. Architecture N-tiers
7. l'Architecture 3-tiers et N-tiers

Vue Physique et Architecture en Couches

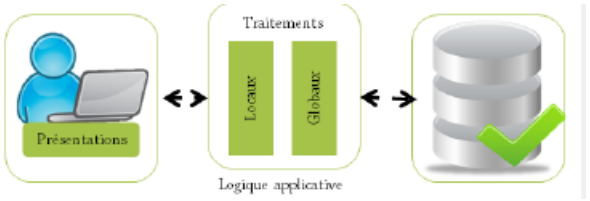
La vue physique d'une architecture logicielle et les choix en couches (architecture client/serveur, 3-tiers, N-tiers) sont étroitement entrelacés de la manière suivante :

- **Architecture en Couches :**
  - Divise le système en composants distincts avec des rôles définis.
  - Chaque couche a des responsabilités spécifiques, comme la logique métier ou l'interface utilisateur.
  - Favorise la maintenance et la réutilisation grâce à la séparation des préoccupations.
  - Le modèle, la logique métier et l'interface utilisateur sont représentatifs dans cette vue.
- **Architecture Client/Serveur, 3-Tiers, N-Tiers :**
  - Gère la distribution et les interactions des composants.
  - L'architecture client/serveur sépare clients et serveurs.
  - L'architecture 3-tiers introduit une couche intermédiaire pour la logique métier.
  - La vue physique, de l'interface utilisateur aux données, est incarnée par ces couches.

Ainsi, la vue physique d'une architecture logicielle trouve sa réalisation à travers les choix en couches, établissant une structure ordonnée pour une exécution efficace et une gestion optimale des ressources.

Architecture en Couches (1/2)

- Une application peut être découpée en **trois niveaux d'abstraction**:
  - Niveau présentation,
  - Niveaux logique applicatif ,
  - Niveau données



- La **couche de présentation** ou IHM (interface Homme Machine) permet l'**intèraction** de l'**application** avec l'**utilisateur**. Ce sont : les saisies au clavier, avec la souris et affichage des information à l'écran.

- La **logique applicative**: décrit les **traitements à réaliser** par l'application pour répondre aux besoins des utilisateurs.
  - **Traitement locaux** : les contrôles effectués au niveau du dialogue avec l'IHM (formulaires, champs, boutons, radio,..)
  - **Traitement globaux** : les règles de l'application, appelées aussi logique métier (Business Logic).

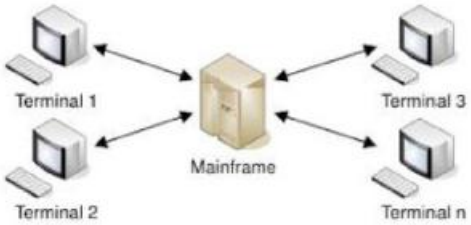
Architecture en Couches (2/2)

- **L'accès aux données** : permet la gestion des informations stockées par l'application. Fonction classiques d'un SGBD:
  - Définition de données
  - Manipulation de données
  - Sécurité de données
  - Gestion des transactions

- Le découpage et répartition de 3 niveaux d'abstraction permettent de distinguer les architectures suivantes:
  - *1-tier, : MainFrame,*
  - *Architecture 2-tiers :client/serveur,*
  - *Architecture 3-tiers: application web*
  - *et Architecture N-tiers*

Architecture 1-tier: Mainframe

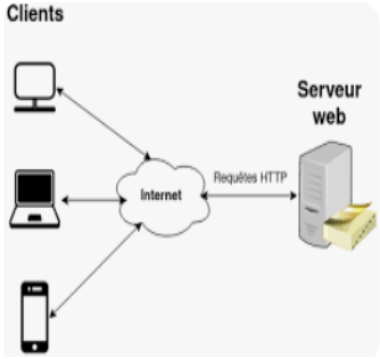
- Les utilisateurs se connectent aux applications exécutées par le **serveur central** (mainframe) à l'aide de **terminaux passifs**.
- Le **serveur central** prend en charge la gestion des **données et des traitements**, y compris l'**affichage** qui est transmis sur des terminaux passifs
- Architecture adoptée durant les années 1970-1980

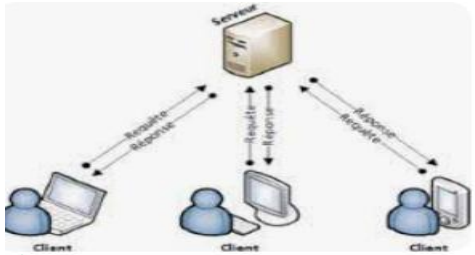



Mainframe AS/400

Architecture 2-tiers: Client/Serveur

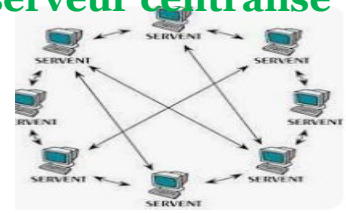
- Dans l'architecture **client/serveur**, (les application sont réparties en **deux types de machines**:
- **Machine Client:**
  - Elle effectue une demande de service auprès du serveur (requêtes)
  - Elle initie le contact (parle en premier), ouvre la session
- **Machine Serveur:**
  - Elle est la partie de l'application qui offre un service
  - Elle est à l'écoute des requêtes clientes
  - Elle répond au service demandé par le client (réponse)





Architecture client/serveur centralisé

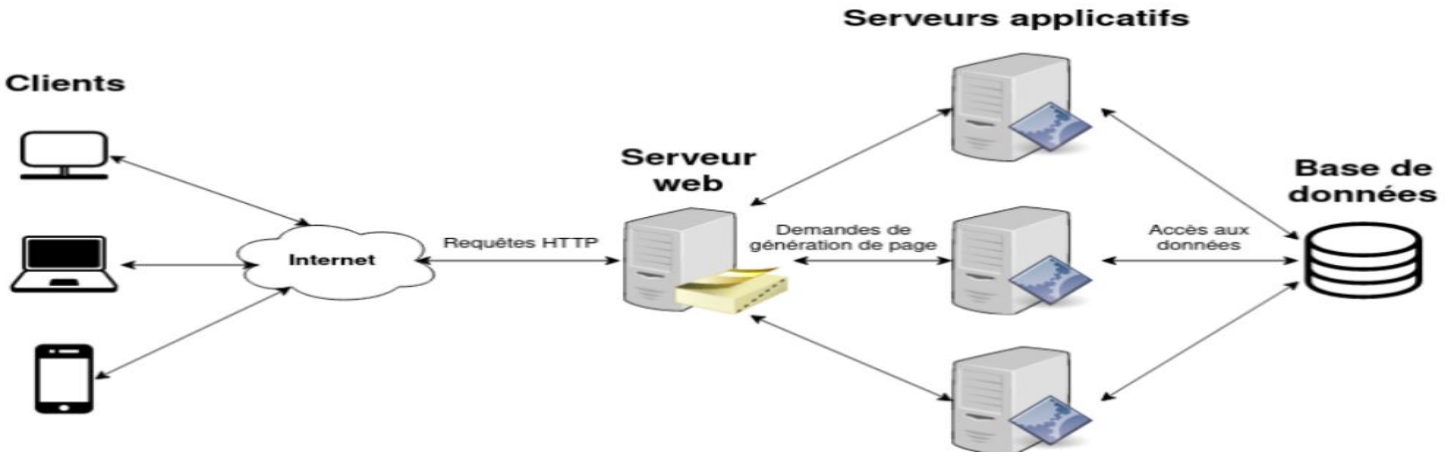
- **Avantages:**
  - Utilisation d'une interface utilisateur riche
  - Appropriation des applications par l'utilisateur
  - Introduction de la notion d'interopérabilité
- **Inconvénients:**
  - Efforts de sécurité
  - Problèmes d'administration
  - Coûts de déploiement et de maintenance



Architecture client/serveur décentralisé (P2P)

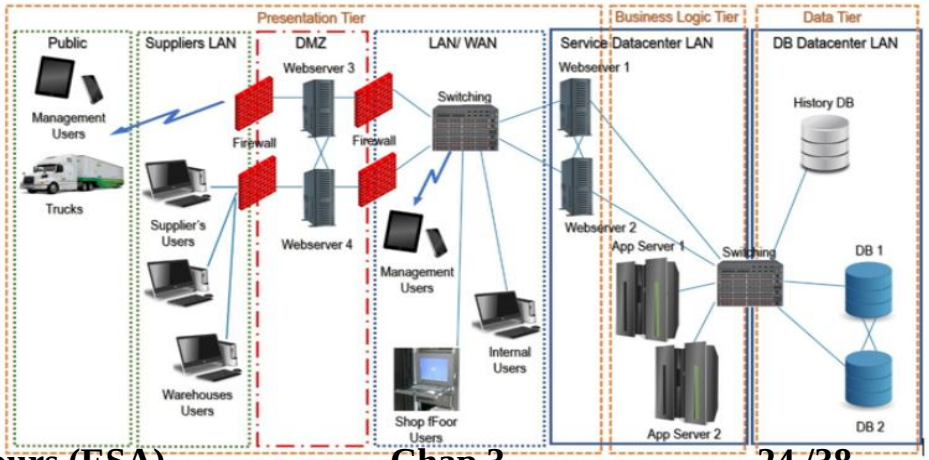
Architecture 3-tiers

- **L'architecture 3-tiers : web dynamique:**
  - **La présentation** est **toujours** prise en charge par le **poste client**
  - La **logique applicative** est prise en charge par un **serveur intermédiaire**: c'est **le serveur applicatif**
  - **Les données** sont toujours **gérées de façon centralisées**. Elles sont gérées par un système de gestion de base de données **(SGBD)**



Architecture N-tiers

- Dans **l'architecture N-tiers**, le **serveur applicatif** de l'architecture 3-tiers **est décomposé** en plusieurs serveurs d'application (par exemple un serveur par région)
  - L'**avantage** de l'architecture N-tiers par rapport à celle 3-tiers est en terme de **performance**. C'est-à-dire, le délai de réponse aux requêtes Clients**.**



l'Architecture 3-tiers et N-tiers

- **Avantages**
  - La poste client ne supporte plus des traitements.
  - **Réduire les coûts** liés au **déploiement** et à la **maintenance**
  - Les **ressources réseau** sont **mieux exploitées**. En effet, les traitements applicatifs sont centralisés
  - La **fiabilité** et les **performances** de certains traitements sont améliorées par leur centralisation
  - Prise en charge simple d'une force montée en charge en renforçant le service applicatif
- **Inconvénients**
  - Une **expertise de développement plus longue** que dans le cadre d'une architecture 2-tiers, est nécessaire
  - Les **coûts de développement** sont plus **élevés** que pour le 2tiers



Institut Supérieur d'Informatique et de Mathématiques de Monastir

Partie 3 : Modèles Architecturaux de la Vue Logique : La Famille MV*

Contenu de la Partie 3

1. Vue Logique et Modèles Architecturaux
2. Modèle-Vue-Contrôleur (MVC)
3. Modèle-vue-présentateur (MVP)
4. Modèle-vue-VueModèle (MVVM)

Vue Logique et Modèles Architecturaux

La vue logique d'une architecture logicielle et le résumé des modèles de conception architecturale sont étroitement liés :

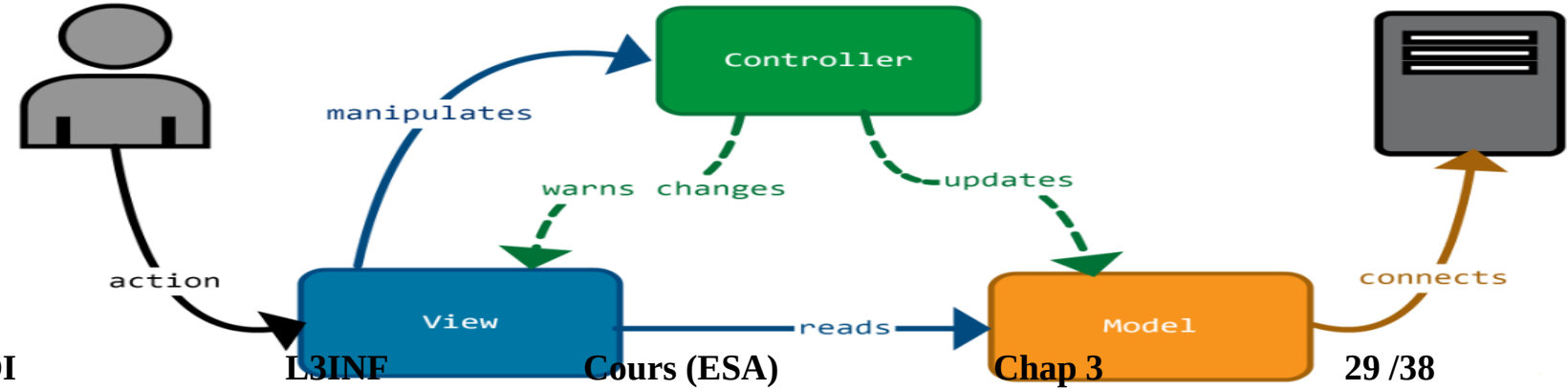
- Les modèles de **conception architecturale** (MVC, MVP, MVVM) offrent une structure claire pour la conception d'applications modernes.
- Ils **facilitent** la **gestion** de la **complexité croissante** du développement.
- Ces modèles **séparent** les **responsabilités** pour créer des applications **testables et maintenables**.
- Chaque modèle propose une **approche unique pour organiser les relations entre le modèle sous-jacent**, **la vue utilisateur et la logique de contrôle ou de présentation.**
- Le choix du modèle dépend du contexte du projet, des besoins et des préférences de développement.
- Ces modèles visent à optimiser la structure et la robustesse de l'application finale.
- En utilisant ces modèles, la vue logique d'une architecture logicielle gagne en clarté et en efficacité, améliorant la qualité globale du développement.

Modèle-Vue-Contrôleur (MVC) (1/3)

- **Modèle-vue-contrôleur** ou **MVC** est un **motif d'architecture logicielle** destiné aux **interfaces graphiques**,
- lancé en 1978 et très populaire pour les **applications web**

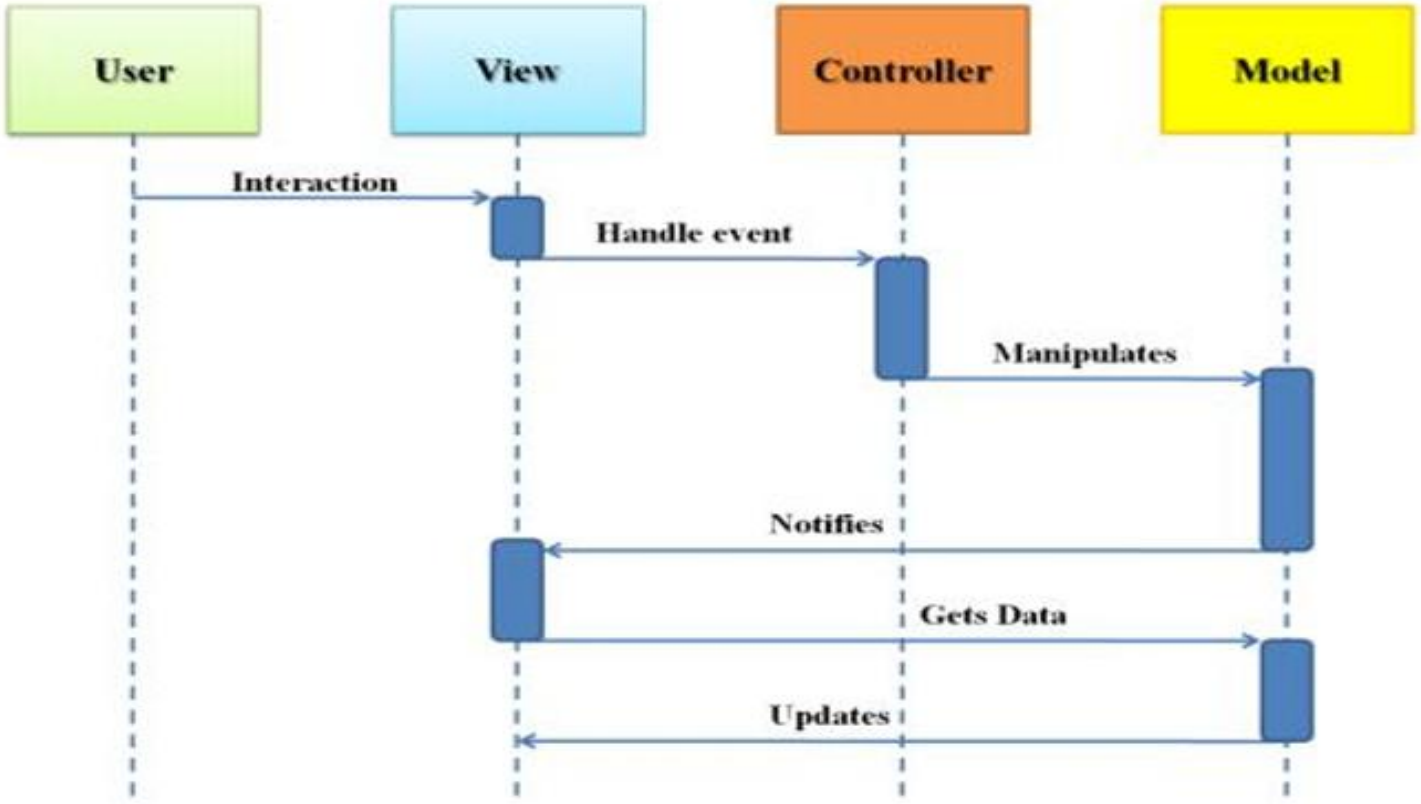
- Le motif est composé de **trois types de modules** ayant **trois responsabilités différentes** :
  - **les modèles**: contient les données à afficher
  - **les vues :**contient la présentation de l'interface graphique
  - et **les contrôleurs :**contient la logique concernant les actions effectuées par l'utilisateur

- MVC est utilisé par de nombreux Frameworks pour applications web tels que Ruby on Rails, Grails, ASP.NET, Spring, Struts, Symfony, Apache Tapestry, Laravel, AdonisJS, Django.



Modèle-Vue-Contrôleur (MVC) (2/3)

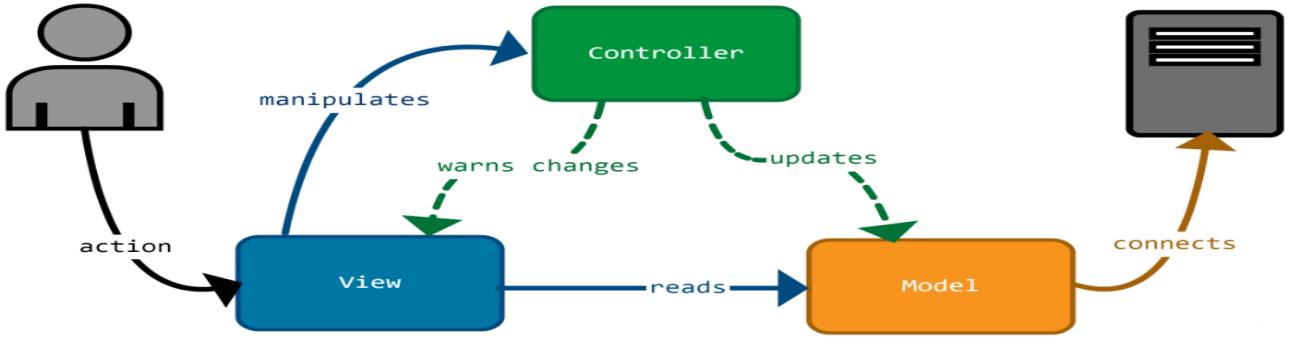
- **Extrait d'un diagramme de séquence selon MVC**



Sequence Diagram of MVC

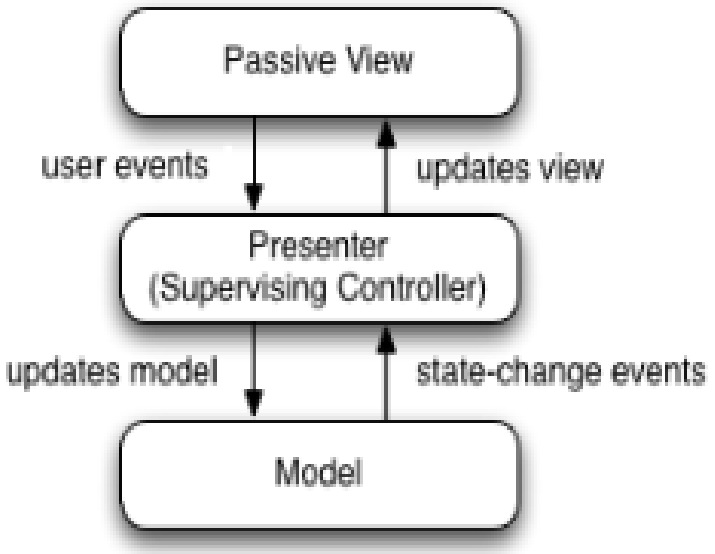
Modèle-Vue-Contrôleur (MVC) (3/3)

- **Avantages :**
  - Séparation des préoccupations (plus ciblée).
  - Facilite le test et la gestion du code.
  - Favorise le découplage des couches de l'application.
  - Meilleure organisation et réutilisabilité du code.
- **Inconvénients :**
  - difficile à comprendre pour les développeurs débutants.
  - la séparation des responsabilités peut entraîner une complexité accrue de l'application;
  - Architecture complexe pour des petits projets;
  - Le nombre important de fichiers représente une charge non négligeable dans un projet.



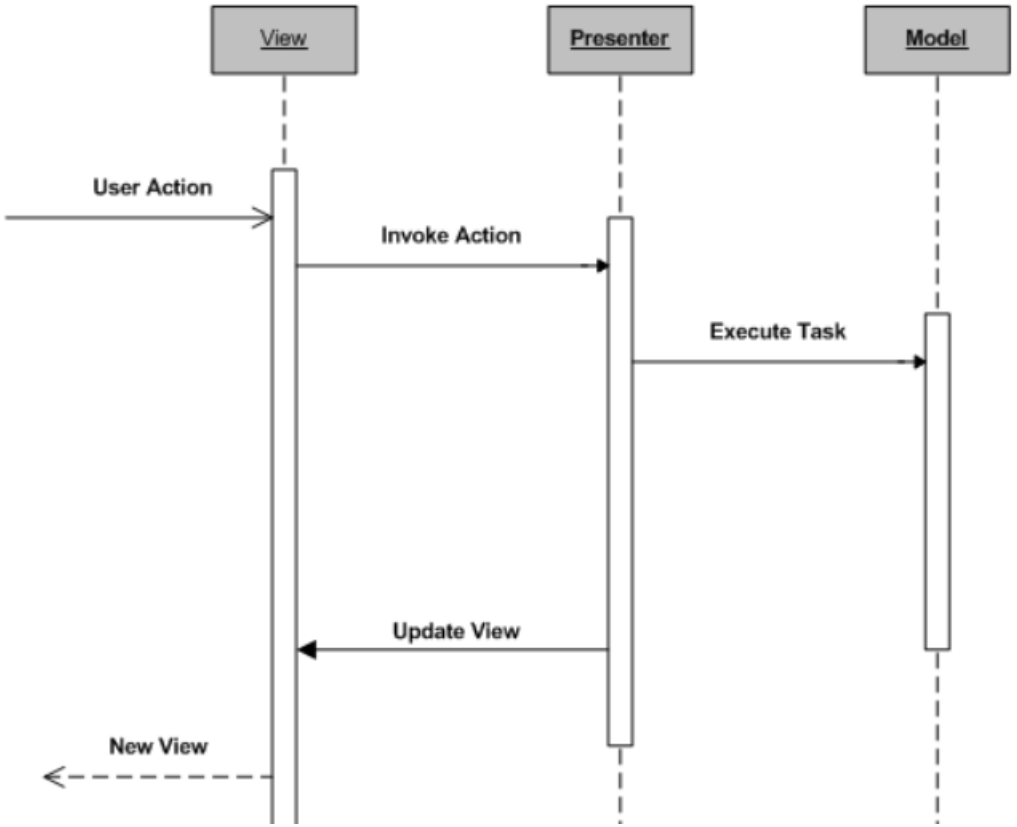
Modèle-vue-présentateur (MVP) (1/3)

- **Modèle-vue-présentateur (MVP)** est une **dérivation** du modèle architectural modèle-vue-contrôleur (**MVC**) et est **principalement utilisé pour créer des interfaces utilisateur.**
- **Il est conçu pour faciliter les tests unitaires automatisés** et **améliorer la séparation des problèmes dans la logique de présentation.**
- Le modèle **MVP** partage deux composants avec MVC : le **modèle et la vue**.
- Il remplace le **contrôleur par le présentateur**.
  - Le **présentateur**, comme son nom l'indique, est utilisé pour présenter quelque chose. Cela vous permet de moquer la vue plus facilement.
  - **Le présentateur** agit sur **le modèle et la vue.** Il récupère les **données** des référentiels **(le modèle)** et les **formate** pour les **afficher dans la vue.**



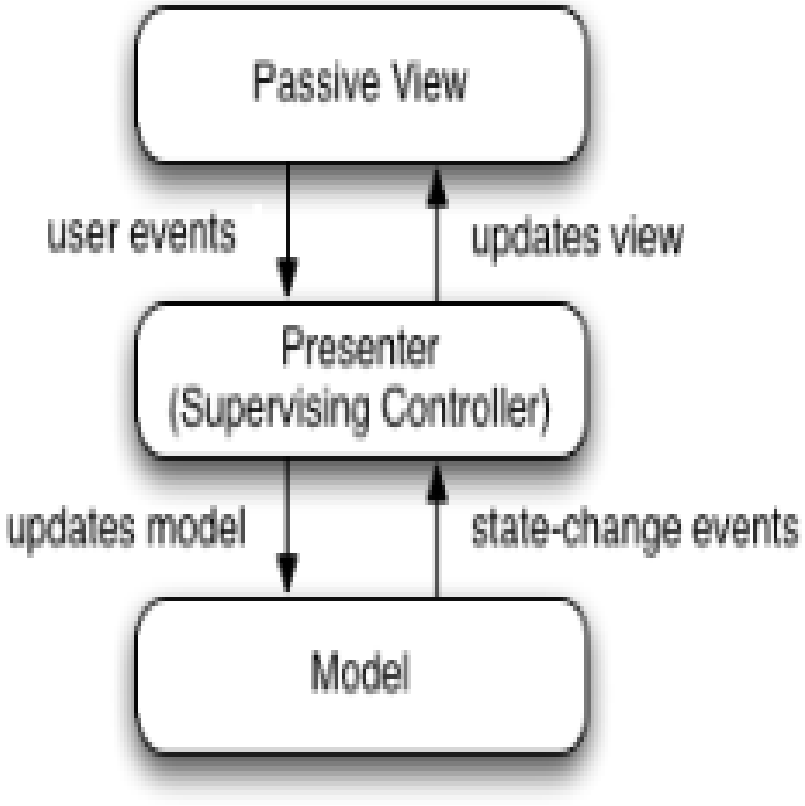
Modèle-vue-présentateur (MVP) (2/3)

- **Extrait d'un diagramme de séquence objet selon MVP**



Modèle-vue-présentateur (MVP) (3/3)

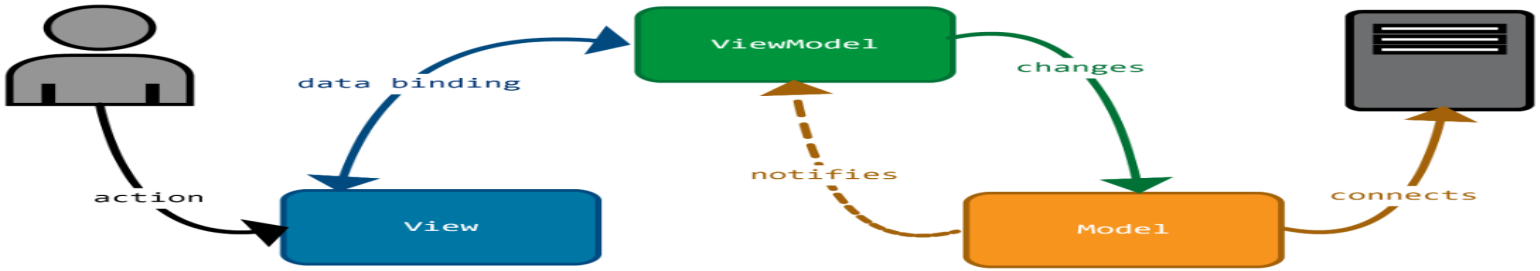
- **Avantages :**
  - testabilité de l'application est largement améliorée
  - toute logique de présentation est en un seul endroit, porter l'application vers une autre bibliothèque graphique est facilité
- **Inconvénients :**
  - Redondance de code dans la présentation



Modèle-vue-VueModèle (MVVM) (1/3)

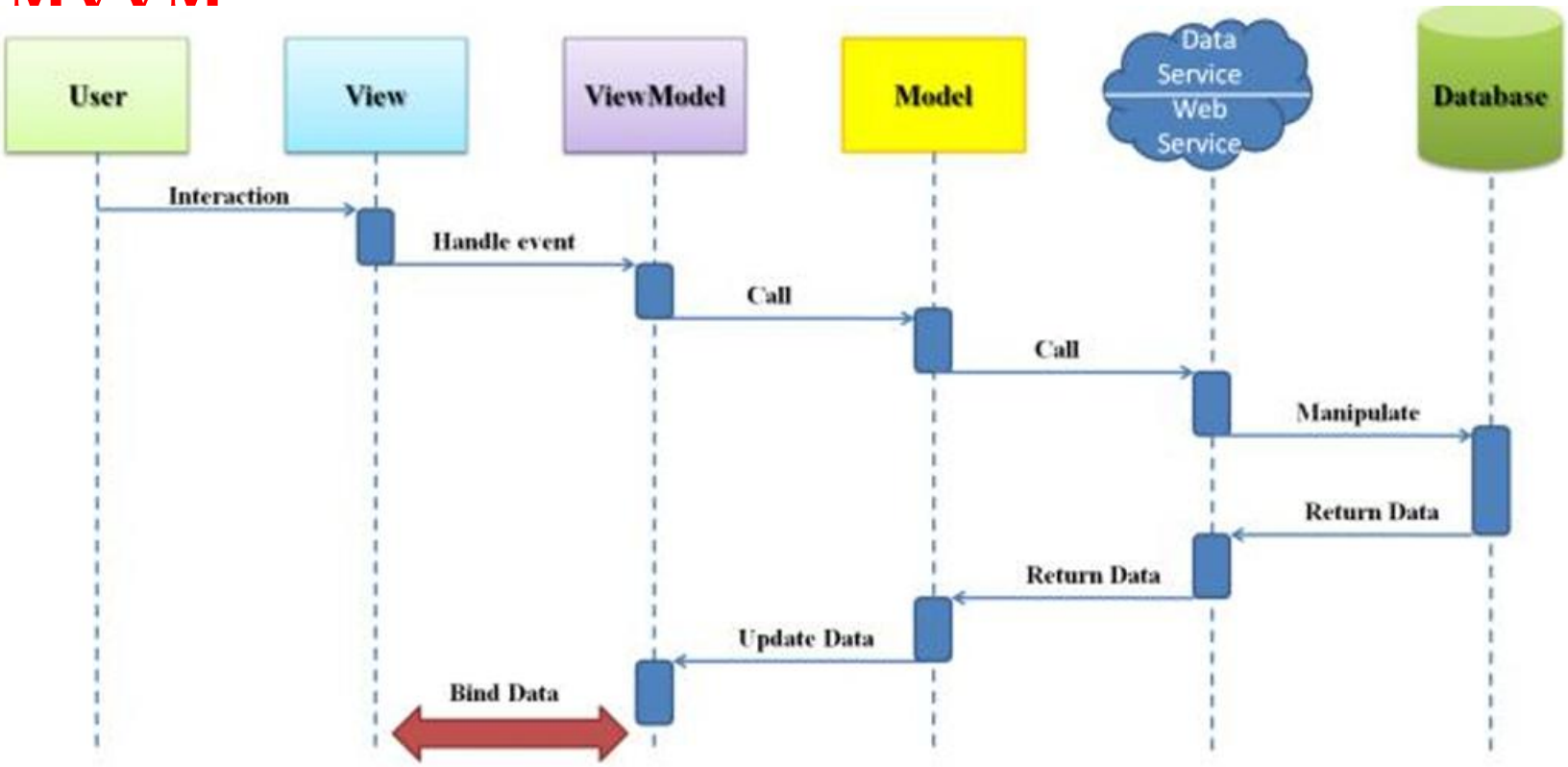
- **Modèle-vue-VueModèle (MVVM)** est fondamentalement le même que MVP, à l'exception d'une différence majeure.
- Le retour d'information n'est plus pris en charge par la présentation mais par un mécanisme de liaison de données (data binding)
- MVVM est l'évolution moderne de MVC.
- L'objectif principal de MVVM est de fournir une séparation claire entre la logique de domaine et la couche de présentation.
- MVVM prend en charge la liaison de données bidirectionnelle entre la vue et le modèle de vue.

- Le modèle MVVM vous permet de séparer la vue et le modèle de votre code. Cela signifie que lorsque le modèle change, la vue n'a pas besoin de le faire, et vice versa. À l'aide d'un modèle de vue, vous pouvez effectuer des tests unitaires et tester votre comportement logique sans impliquer votre vue.



Modèle-vue-VueModèle (MVVM) (1/2)

- **Extrait d'un diagramme de séquence selon MVVM**

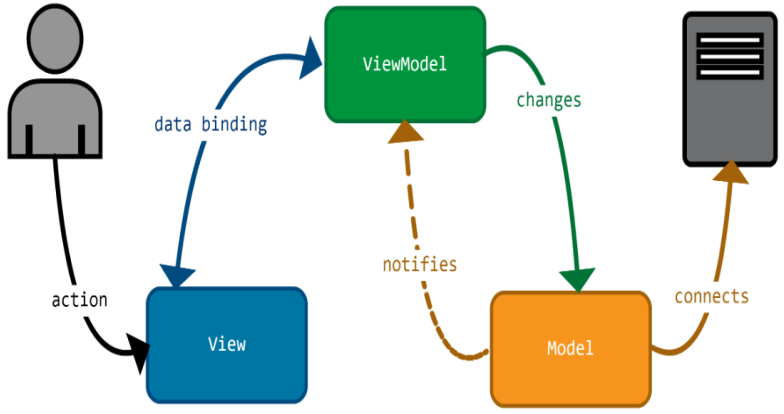


Sequence Diagram of MVVM

Modèle-vue-Vue Modèle (MVVM) (3/3)

- **Avantages :**
  - Les développeurs peuvent créer des tests unitaires pour le modèle de vue et le modèle, sans utiliser la vue.
  - L'IU de l'application peut être repensée sans toucher au modèle de vue et au code du modèle
  - Les concepteurs et les développeurs peuvent travailler indépendamment et simultanément sur leurs composants durant le développement. Les concepteurs peuvent se concentrer sur la vue, tandis que les développeurs peuvent travailler sur le modèle de vue et les composants du modèle.

- **Inconvénients :**
  - Complexité accrue
  - Apprentissage
  - Ressources supplémentaires
  - Surcharge de liaison de données
  - Comptabilité



Fin

aljia.bouzidi95@gmail.com